

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики»**



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

по дисциплине

«Функциональная схемотехника»:

Вариант 4

Выполнили:

Ястребов-Амирханов Алекси

Исупов Никита Александрович

Поток: 1.1

ПРЕПОДАВАТЕЛЬ: Табунщик Сергей Михайлович

Санкт-Петербург,

2025

Цель работы:

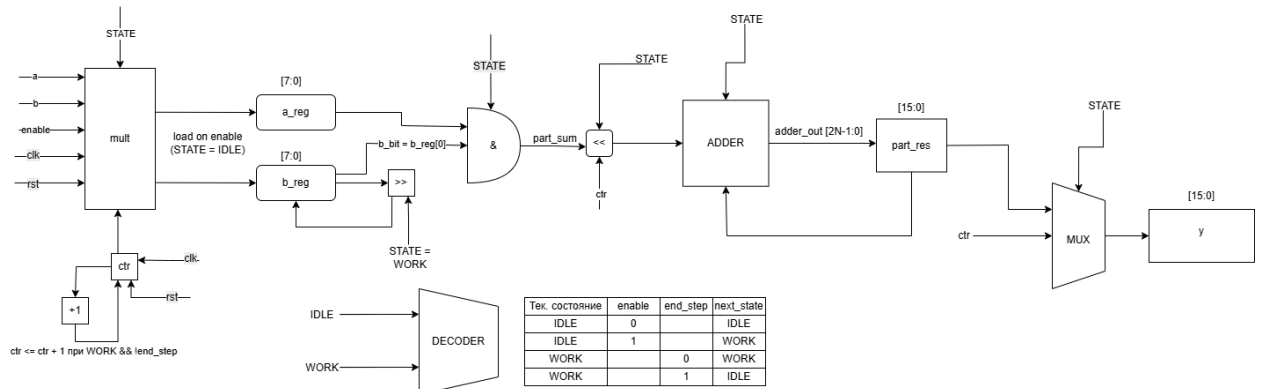
Получить навыки описания арифметических блоков на RTL-уровне с использованием языка описания аппаратуры Verilog HDL.

Задание по варианту:

№ варианта	Функция	Ограничения
4	$y = \sqrt{a + \sqrt[3]{b}}$	2 сумматора и 1 умножитель

Схему (рисунок) разработанного блока

Модуль mult



Описание работы модуля mult

Разработанный модуль mult реализует последовательное умножение двух N-разрядных чисел a и b с помощью пошагового суммирования частичных произведений. Работа блока управляется сигналами clk, rst и enable.

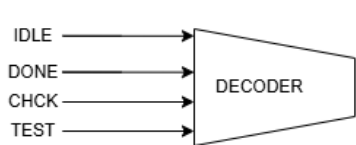
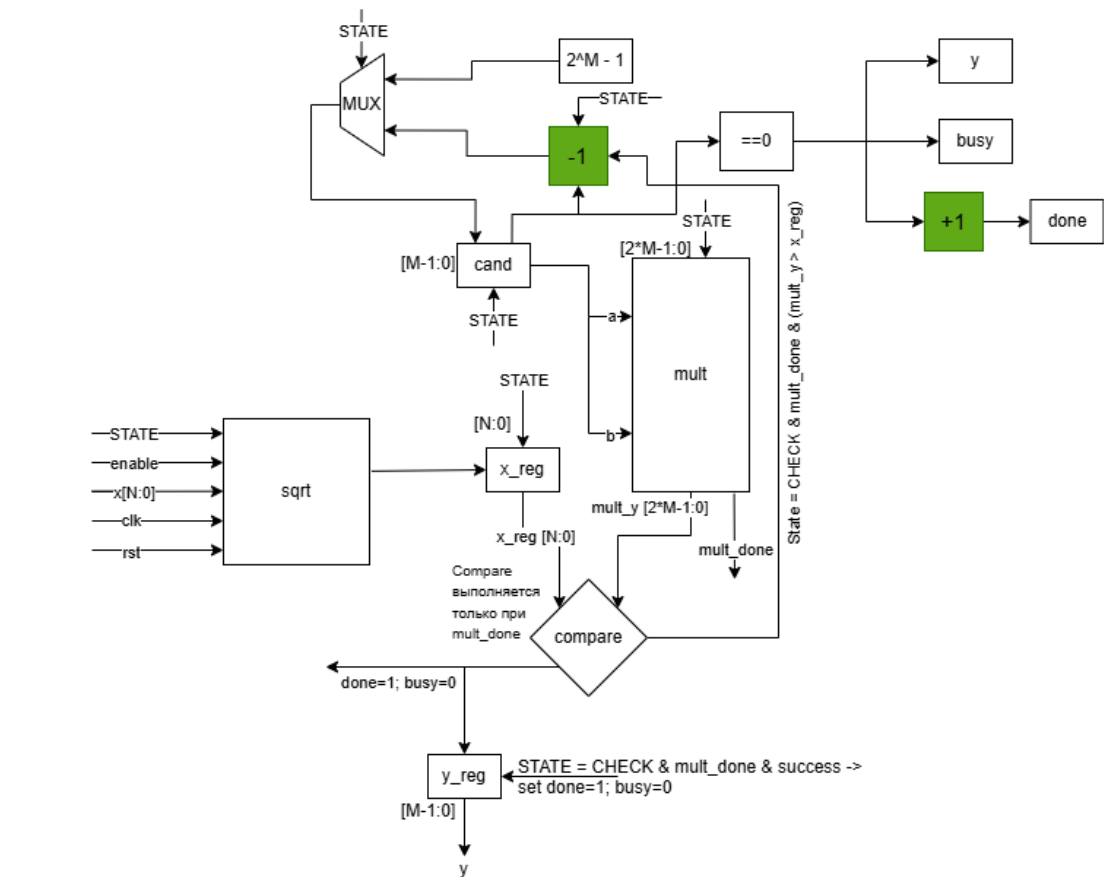
- Инициализация (сброс):**
При активном сигнале rst все внутренние регистры и выходы (ctr, part_res, y, a_reg, b_reg, done, state) обнуляются. Блок переходит в состояние IDLE.
- Начало операции:**
Когда enable = 1 в состоянии IDLE, входные данные a и b записываются во внутренние регистры a_reg и b_reg. Счетчик ctr и промежуточный результат part_res обнуляются. Блок переходит в состояние WORK, а сигнал busy становится равным 1.
- Процесс умножения:**
В каждом такте происходит проверка младшего бита b_reg[0].
 - Если он равен 1, частичное произведение a_reg добавляется к part_res с соответствующим сдвигом (<< ctr).

- Затем `b_reg` сдвигается вправо на один бит, а `ctr` увеличивается на единицу.
4. Завершение:
- Когда `ctr` достигает значения $N - 1$, вычисляется последний частичный результат. Итоговое произведение записывается в выход `y`, сигнал `done` устанавливается в 1, а состояние возвращается в `IDLE`. После этого `busy` снова становится 0, и модуль готов к следующему запуску.

Область допустимых значений

- Разрядность входных данных: `a`, `b` — **N бит** (параметр модуля).
- Диапазон значений: от **0** до $2^n - 1$.
- Сигналы `clk`, `rst`, `enable` — **логические** (0 или 1).
- При `rst = 1` выполняется сброс, при `enable = 1` запускается операция.
- Выход `y` имеет разрядность **$2N$ бит**, диапазон значений — от **0** до $(2^n - 1)^2$.

Модуль sqrt



Curr State	enable	mult_done	comp	cand_zero	Next State	Next mult	Next busy	Next done
IDLE	0				IDLE	0	0	0
IDLE	1				TEST	0	1	0
TEST					CHECK	1	1	0
CHECK		0			CHECK	0	1	0
CHECK		1	1		DONE	0	0	1
CHECK		1	0	1	DONE	0	0	1
CHECK		1	0	0	TEST	0	1	0
DONE	1				DONE	0	0	1
DONE	0				IDLE	0	0	0
(default)					IDLE	0	0	0

Описание работы модуля sqrt

Модуль sqrt вычисляет целую часть квадратного корня из входного числа x (шириной N+1 бит), используя вспомогательный модуль mult для возведения кандидата cand в квадрат.

- 1. Сброс:
При rst = 1 все регистры и сигналы (state, y, x_reg, cand, done, busy) обнуляются. Модуль переходит в состояние IDLE.
- 2. Запуск вычисления:
При enable = 1 во входных данных, число x сохраняется во внутренний регистр x_reg, а кандидат cand инициализируется максимальным

возможным значением ($2^M - 1$). Затем начинается процесс проверки — переход в состояние TEST.

3. Проверка кандидата:

В состоянии TEST запускается модуль mult, который вычисляет $\text{cand} * \text{cand}$. После завершения умножения ($\text{mult_done} = 1$) модуль сравнивает результат с x_reg .

4. Сравнение и выбор результата:

- Если $\text{cand}^2 \leq \text{x_reg}$, найдено подходящее значение, оно записывается в y , устанавливается $\text{done} = 1$, и вычисление завершается.
- Если $\text{cand}^2 > \text{x_reg}$, cand уменьшается на 1, и процесс повторяется до нахождения наибольшего целого кандидата, квадрат которого не превышает x .

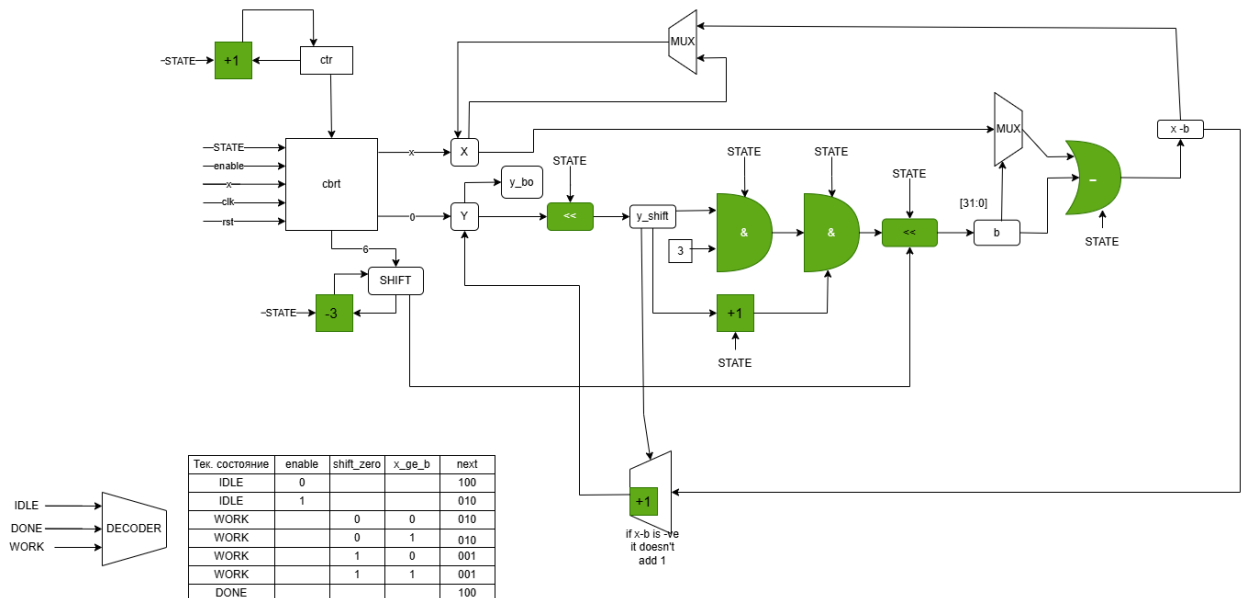
5. Завершение:

После установки $\text{done} = 1$ модуль ожидает сброса enable для возврата в состояние IDLE и готовности к новому запуску.

Область допустимых значений

- Вход x : $(N+1)$ -битное неотрицательное число, диапазон — от 0 до $2^{(N+1)} - 1$.
- Сигналы clk , rst , enable : логические (0 или 1).
- Параметр N — чётное положительное число (например, 8).
- Выход y : целая часть квадратного корня, диапазон от 0 до $2^M - 1$, где $M = N/2 + 1$.

Модуль cbrt



Описание работы модуля cbrt

Модуль `cbqrt` вычисляет целую часть кубического корня из входного числа `x` шириной `N` бит. Алгоритм реализован в виде итерационного побитового метода, работающего в несколько тактов.

1. Сброс:
При $\text{rst} = 1$ все регистры и сигналы (state , y , x_reg , y_reg , shift , done , busy) обнуляются, и модуль переходит в состояние IDLE.
2. Запуск вычисления:
При $\text{enable} = 1$ значение x загружается во внутренний регистр x_reg (расширенный до двойной ширины), обнуляется промежуточный результат y_reg , вычисляется начальное значение shift , и модуль переходит в состояние WORK.
Сигнал busy устанавливается в 1.
3. Пошаговое вычисление:
В каждом цикле:
 - Формируется промежуточная величина $y_shift = y_reg \ll 1$.
 - Вычисляется проверочное значение $b = (3 * y_shift * (y_shift + 1) + 1) \ll \text{shift}$.
 - Если $x_reg \geq b$, из x_reg вычитается b , а y_reg увеличивается на 1.
Иначе значение y_reg сохраняется без изменений.

- Сдвиг shift уменьшается на 3, и процесс повторяется до тех пор, пока shift не станет равен нулю.

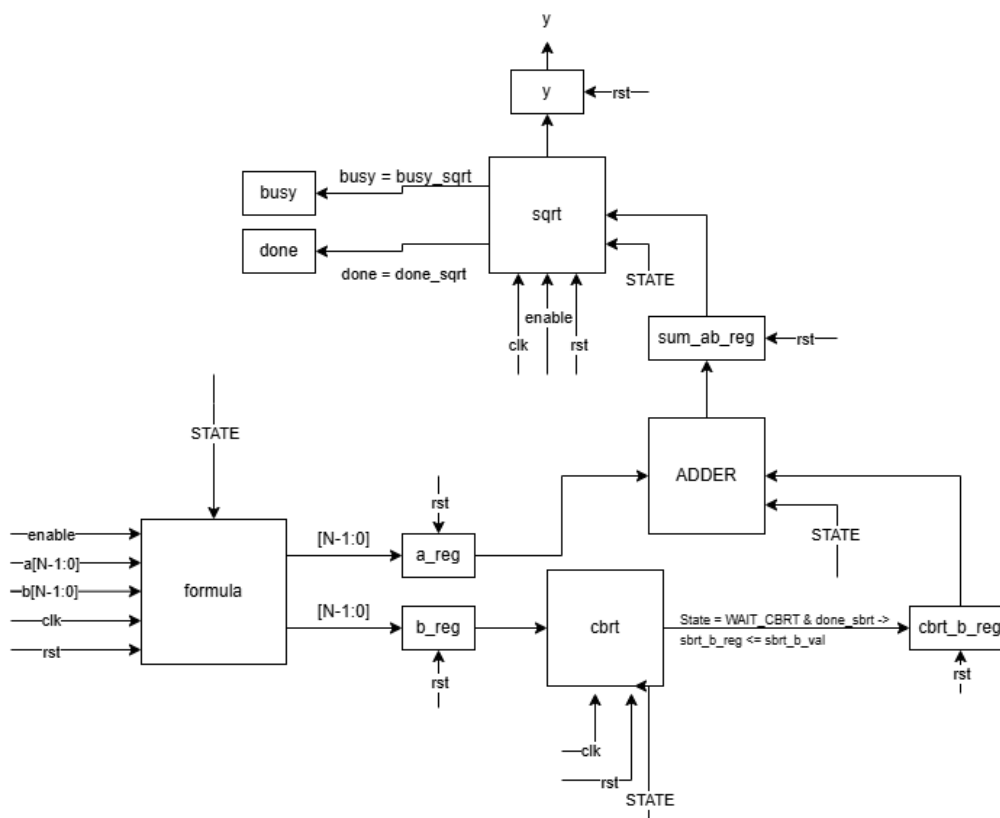
4. Завершение:

После завершения итераций результат записывается в выход y (младшие N бит из y_reg), устанавливается $done = 1$, $busy = 0$, и модуль возвращается в состояние IDLE, готовый к новому запуску.

Область допустимых значений

- Вход x : N -битное неотрицательное число, диапазон — от 0 до $2^n - 1$.
- Параметр N : положительное число, обычно кратное 3 (для корректности сдвигов).
- Сигналы clk , rst , $enable$: логические (0 или 1).
- Выход y : целая часть кубического корня, диапазон от 0 до $\lfloor (2^n - 1)^{(1/3)} \rfloor$.

Модуль formula



Модуль `formula` вычисляет выражение с использованием встроенных подмодулей `cbrt` (кубический корень) и `sqrt` (квадратный корень). Работа выполняется поэтапно, под управлением конечного автомата (FSM).

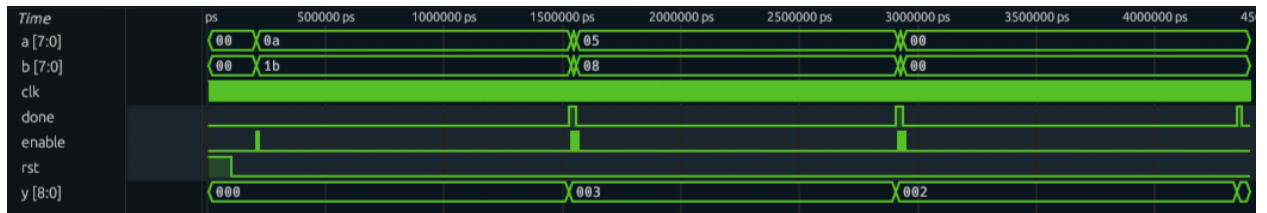
1. Сброс:
При $rst = 1$ все регистры (a_reg , b_reg , $sbrt_b_reg$, sum_ab_reg) и сигналы (y , $done$, $state$, $enable_sbrt$, $enable_sqrt$) обнуляются. Модуль переходит в состояние IDLE.
2. Запуск вычисления:
При $enable = 1$ входные значения a и b записываются во внутренние регистры, и модуль переходит в состояние START_CBRT для начала вычисления кубического корня.
3. Вычисление кубического корня:
 - В состоянии START_CBRT посылается импульс $enable_sbrt = 1$ для запуска подмодуля $cbrt$.
 - В состоянии WAIT_CBRT модуль ожидает сигнала $done_sbrt = 1$.
 - После завершения вычисления результат $cbrt(b)$ сохраняется в $sbrt_b_reg$, и вычисляется сумма $a + cbrt(b)$.
4. Вычисление квадратного корня:
 - В состоянии START_SQRT подается импульс $enable_sqrt = 1$ для запуска подмодуля $sqrt$.
 - В WAIT_SQRT происходит ожидание завершения ($done_sqrt = 1$).
 - Полученный результат $sqrt(a + cbrt(b))$ записывается в y .
5. Завершение:
После получения результата устанавливается $done = 1$, модуль переходит в состояние DONE_STATE, а затем возвращается в IDLE, ожидая нового запуска ($enable$ снова должен быть сброшен).

Область допустимых значений

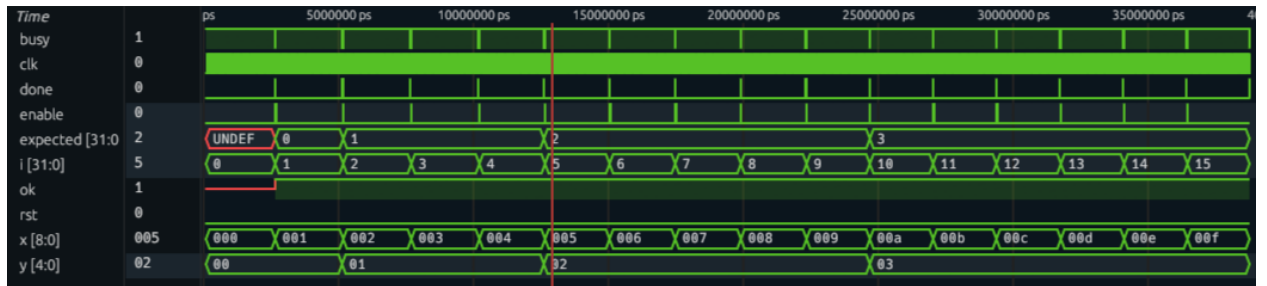
- Входы a , b : N -битные неотрицательные числа, диапазон — от 0 до $2^n - 1$.
- Сигналы clk , rst , $enable$: логические (0 или 1).
- Выход y : N -битный результат, равный целой части выражения $sqrt(a + cbrt(b))$.
- Параметр N — положительное целое число (например, $N = 8$).

Результат работы разработанного блока

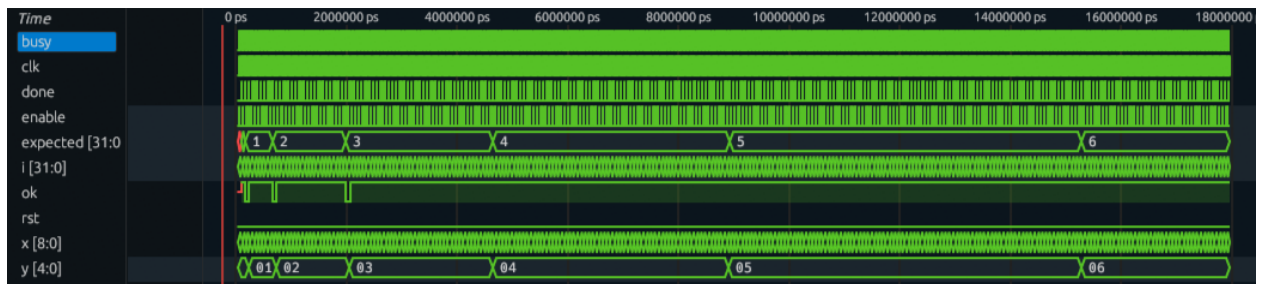
Formula



Sqrt



Cbirt



Вывод:

В процессе лабораторной работы мы поработали с программой Vivado и с расширением для VSCode. Ознакомились с основными компонентами Верилонг. Проекты в Vivado нас не прекращают удивлять. До сих пор не понятно, как правильно удалять файлы из самого приложения. Так как они постоянно сами подтягиваются даже если нажать delete. Но решение было найдено. Просто при любой ошибке, пересоздавать проект. Так мы научились смирению и перепроверке всего 100 раз. Так же мы построили нужную по нашему варианту формулу, на заданных ограничениях. Провели тестирования, и определили ОДЗ наших модулей.